Listat dhe Tuplet në Python

Alion Emini

Kurs i Programimit në Python
kurse.informatika
September 30, 2025

Contents

1 Listat dhe Tuplet **5**

1.1 Sekuenca (Sequences) . . . 5

1.2 Hyrje në Lista . . . 6

1.2.1 Shfaqja e një liste të plotë . . . 6

1.2.2 Funksioni i brendshëm `list()` . . . 7

1.3 Operatori i Përsëritjes (`*`) . . . 7

1.4 Iterimi mbi një listë me ciklin `for` . . . 8

1.5 Indeksimi (Indexing) . . . 9

1.5.1 Indekse negative . . . 10

1.5.2 Gabimi `IndexError` kur indeksi është i pavlefshëm . . . 10

1.6 Funksioni `len` . . . 10

1.7 Përdorimi i `for` për të iteruar sipas indeksit . . . 11

1.8 Listat janë *mutable* (të ndryshueshme) . . . 11

1.8.1 Mbushja e një liste me indekse . . . 12

1.9 Bashkimi (konkatenimi) i listave . . . 13

1.9.1 Me + (krijon listë të re) . . . 13

1.9.2 Me += (shton në vend) . . . 13

1.10 Prerja e listave (List Slicing) . . . 14

1.10.1 Shembuj të shkurtër me numra . . . 14

1.10.2 Lënia bosh e `start` ose `end` . . . 15

1.10.3 Hapi i prerjes (opsional) . . . 15

1.10.4 Indekse negative në slicing . . . 16

1.11 Kërkimi i elementëve në lista me operatorin `in` . . . 16

1.12 Kopjimi i listave — pesë mënyra . . . 17

1.12.1 Pse `list1 = list2` **nuk** është kopjim? . . . 17

1.12.2 Mënyra 1 — `list.copy()` . . . 18

1.12.3 Mënyra 2 — `list( iterable )` . . . 18

1.12.4 Mënyra 3 — `[:]` (slicing i plotë) . . . 19

1.12.5 Mënyra 4 — `for` + `append` . . . 19

1.12.6 Mënyra 5 — `[] + list` . . . 19

1.13 Përpunimi i listave . . . 20

1.13.1 Shuma e vlerave në një listë . . . 20

1.13.2 Mesatarja e vlerave në një listë . . . 21

1.13.3 Kalimi i një liste si argument funksioni . . . 21

1.13.4 Kthimi i një liste nga një funksion . . . 22

1.14 Përzgjedhja e rastësishme e elementëve të një liste . . . 23

1.14.1 `random.choice` — një element i vetëm . . . 23

1.14.2 `random.choices` — disa elementë (me përsëritje të mundshme) . . . 23

1.14.3 `random.sample` — elementë unikë (pa përsëritje) . . . 23

1.15 Punë me lista dhe skedarë . . . 23
1.15.1 Shkrimi i një liste *string*-ësh: `writelines` . . . 24
1.15.2 Shkrimi i një elementi në çdo rresht . . . 24
1.15.3 Leximi në listë me `readlines` dhe `rstrip` . . . 25
1.15.4 Shkrimi i numrave në skedar . . . 26
1.15.5 Leximi i numrave dhe konvertimi në `int` . . . 26
1.16 List Comprehensions (Përftimi i listave) . . . 27
1.16.1 Pse t'i përdorim? . . . 27
1.16.2 Shembuj transformimi . . . 28
1.16.3 Filtrim me klauzolën `if` . . . 28
1.17 Metodat e listave dhe funksione të dobishme . . . 30
1.17.1 `append` — shto në fund . . . 30
1.17.2 `index` — gjej pozicionin e një elementi . . . 30
1.17.3 `insert` — fut në pozicion dhe zhvendos djathtas . . . 31
1.17.4 `sort` — rendit në rritje . . . 31
1.17.5 `remove` — hiq paraqitjen e parë . . . 32
1.17.6 `reverse` — kthe rendin mbrapsht . . . 32
1.17.7 `del` — fshi sipas indeksit . . . 33
1.17.8 `min` dhe `max` — vlerat skajore . . . 33
1.18 Listat dy-dimensionale (Two-Dimensional Lists) . . . 34
1.18.1 Shembull i shpejtë: listë me emra (3×2) . . . 34
1.18.2 Indeksimi me dy nënindekse . . . 34
1.18.3 Iterimi: dy `for`-cikla të folezuar (row-major) . . . 35
1.18.4 Caktimi i vlerave me indekse . . . 36
1.19 Tuplet . . . 36
1.19.1 Krijimi dhe shfaqja . . . 37
1.19.2 Iterimi me `for` . . . 37
1.19.3 Indeksimi dhe `range(len(...))` . . . 37
1.19.4 Çfarë mbështesin tuplet . . . 38
1.19.5 Çfarë *nuk* mbështesin tuplet . . . 38
1.19.6 Tuple me një element . . . 38
1.19.7 Pse të përdorim tuplet? . . . 38
1.19.8 Konvertimi mes listave dhe tupleve . . . 38
1.19.9 Shembuj përdorimi të tupleve . . . 39

Chapter 1

Listat dhe Tuplet

Objektivat e kapitullit

- Të kuptosh çfarë është *sekuencë* dhe dallimin **listë** (*mutable*) vs **tuple** (*immutable*).
- Të krijosh dhe të shfaqësh lista/tuples; të përdorësh `len()`, indeksimin (edhe indekse negative) dhe iterimin me `for`/`while`.
- Të përdorësh **slicing** `list[start:end:step]` dhe rregullat e kufijve (start/end bosh, hap, indekse negative).
- Të aplikosh operatorët `+`, `+=`, `*` (përsëritja), `in` / `not in`.
- Të **kopjosh** lista (5 mënyra) dhe të kuptosh "*shallow copy*" dhe pasojat kur elementët janë objekte të përbëra.
- Të përpunosh lista: **shuma**, **mesatarja**, `min` / `max`; të kalosh/kthesh lista nga funksionet.
- Të përdorësh **list comprehensions** (transformim, filtër me `if`, kusht në rezultatin e elementit).
- Të punosh me **skedarë**: `writelines`, `write` / `print(..., file=...)`, `readlines + rstrip`, si edhe konvertime `int`/`float`.
- Të ndërtosh dhe manipulosh **lista 2D** (adresimi `[r][c]`, iterimi *row-major*) dhe të shmangësh kurthin `[[0]*COLS]*ROWS`.
- Të njohësh gabimet e zakonshme: `IndexError`, `ValueError`, `ZeroDivisionError` — dhe si t'i parandalosh.

1.1 Sekuenca (Sequences)

Përkufizim

Sekuencë quhet një objekt që mban disa elemente të dhënash, të ruajtura *njëri pas tjetrit*. Mbi një sekuencë mund të kryhen operacione për të ekzaminuar dhe manipuluar elementët e saj.

Një sekuencë përmban shumë elemente të dhënash të renditura. Python ofron mënyra të ndryshme për të punuar me elementët që ndodhen në një sekuencë. Ka disa lloje objektesh sekuencë në Python. Këtu shohim dy më themeloret: **listat** dhe **tuples**. Të dyja mund të mbajnë tipe të ndryshme të dhënash. Dallimi është i thjeshtë:

- **Listat** janë *mutable* (të ndryshueshme): programi mund të ndryshojë përmbajtjen e tyre.
- **Tuples** janë *immutable* (të pandryshueshme): pasi krijohen, përmbajtja e tyre *nuk* mund të ndryshohet.

1.2 Hyrje në Lista

Përkufizim

Listë është një objekt që përmban disa elemente të dhënash. Listat janë *mutable*, pra përmbajtja e tyre mund të ndryshohet gjatë ekzekutimit të programit. Janë struktura dinamike: elemente mund t'u *shtohen* ose *hiqen*. Mund të përdorësh *indeksim*, *slicing* dhe metoda të ndryshme për të punuar me listat.

Çdo **element** në listë quhet *element*. Shembull që krijon një listë me numra të plotë:

```
even_numbers = [2, 4, 6, 8, 10]
```

Elementët janë brenda kllapave katrore dhe të ndarë me presje. Pas ekzekutimit, variabla `even_numbers` referon listën, si në skemën më poshtë.

2	4	6	8	10

Figure 1.1: Një listë me numra të plotë.

Shembull tjetër:

```
names = ['Molly', 'Steven', 'Will', 'Alicia', 'Adriana']
```

Pas ekzekutimit, variabla `names` referon listën si në Fig. 1.2.

Molly	Steven	Will	Alicia	Adriana

Figure 1.2: Një listë me *string*-je.

Një listë mund të mbajë elemente me tipe të ndryshme:

```
info = ['Alicia', 27, 1550.87]
```

Pas ekzekutimit, `info` referon listën si në Fig. 1.3.

Alicia	27	1550.87

Figure 1.3: Një listë me tipe të ndryshme (string, int, float).

1.2.1 Shfaqja e një liste të plotë

Mund të përdorësh `print` për të shfaqur të gjithë listën:

```
numbers = [5, 10, 15, 20]
print(numbers)
```

Ky `print` do të shfaqë:

```
[5, 10, 15, 20]
```

1.2.2 Funksioni i brendshëm `list()`

Python ofron `list()` për të kthyer objekte të caktuara në lista. Kujto nga Kapitulli 4 që `range` kthen një *iterable* (objekt mbi të cilin mund të iterosh). Mund të përdorësh një deklarim si ky për ta kthyer rezultatin e `range` në listë:

```
numbers = list(range(5))
```

Kur ekzekutohet kjo deklaratë, ndodhin këto:

1) Thirret `range` me argumentin `5`. Funksioni kthen një iterable me vlerat `0, 1, 2, 3, 4`.
2) Ky iterable i jepet `list()` si argument. `list()` kthen listën `[0, 1, 2, 3, 4]`.
3) Lista `[0, 1, 2, 3, 4]` i caktohet variablës `numbers`.

Shembull tjetër:

```
numbers = list(range(1, 10, 2))
```

Kujto nga kapitulli i funksioneve kur `range` merr tre argumente, i pari është *vlera fillestare*, i dyti është *kufiri përfundimtar* (jo i përfshirë), dhe i treti është *hapi*. Kjo deklaratë do t'i caktojë variablës `numbers` listën `[1, 3, 5, 7, 9]`.

1.3 Operatori i Përsëritjes (*)

Kemi mësuar se shenja * shumëzon dy numra. Por kur operandi majtas është një *sekuencë* (p.sh. listë) dhe operandi djathtas është një *numër i plotë*, * vepron si **operator përsëritjeje**. Ky operator krijon disa kopje të listës dhe i bashkon në një të vetme.

```
list * n
```

ku `list` është një listë dhe `n` është numri i kopjeve që do të krijohen. Shembull në sesion interaktiv:

```
1 numbers = [0] * 5 Enter
2 print(numbers) Enter
```

```
[0, 0, 0, 0, 0]
```

- Në rreshtin 1, shprehja `[0] * 5` krijon pesë kopje të listës `[0]` dhe i bashkon në një listë të vetme. Lista që rezulton i caktohet variablës `numbers`.
- Në rreshtin 2, variabla `numbers` i jepet funksionit `print`.

```
numbers = [1, 2, 3] * 3 Enter
(numbers) Enter
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Shënim

Shumica e gjuhëve të programimit ofrojnë struktura sekuencash të quajtura *arrays*, të ngjashme me listat, por shumë më të kufizuara në aftësi. Në Python nuk krijon *arrays* tradicionale sepse listat shërbejnë për të njëjtin qëllim dhe ofrojnë më shumë mundësi të integruara.

1.4 Iterimi mbi një listë me ciklin `for`

Një nga mënyrat më të lehta për të aksesuar elementët individualë të një liste është `for`-cikli.

```
for variable in list:
    statement
    statement
    # etj.
```

Në këtë format, `variable` është emri i një variable, dhe `list` është emri i listës. Sa herë që cikli iteron, `variable` do të referojë një *kopje* të një elementi në listë, duke filluar nga i pari. Themi se cikli *iteron mbi elementët* e listës. Shembull:

```
numbers = [1, 2, 3, 4]
for num in numbers:
    print(num)
```

Variabla `numbers` referon një listë me katër elementë, prandaj ky cikël do të iteron katër herë. Herën e parë `num` do të referojë vlerën `1`, herën e dytë `2`, e kështu me radhë. Kur kodi ekzekutohet, shfaqet:

```
1
2
3
4
```

Figura 1.4 ilustron si `num` referon një *kopje* të një elementi nga lista `numbers` gjatë iterimit. Është e rëndësishme të kuptosh se **nuk** mund të përdorësh `num` për të ndryshuar përmbajtjen e elementit në listë. Nëse ndryshon vlerën që `num` referon brenda ciklit, kjo *nuk* ndikon listën.

```
for num in numbers:
    num = 99
print(numbers)
```

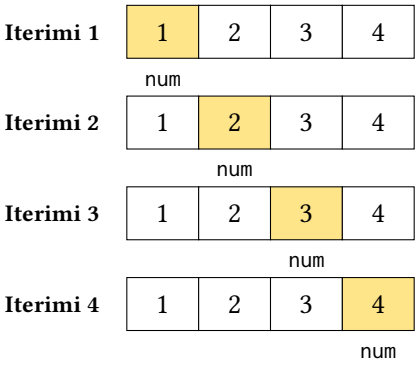

Figure 1.4: Iterimi mbi listën `[1, 2, 3, 4]`: në çdo hap `num` merr nga një element me radhë.

Deklarata në rreshtin 3 thjesht i ricakton variablës `num` vlerën `99` në çdo iterim. Kjo nuk ka asnjë efekt mbi listën e referuar nga `numbers`. Kur ky kod ekzekutohet, do të printohet:

```
[1, 2, 3, 4]
```

1.5 Indeksimi (Indexing)

Çdo element në një listë ka një *indeks* që tregon pozicionin e tij në listë. Indeksimi fillon nga `0`, pra elementi i parë ka indeks `0`, i dyti `1`, e kështu me radhë. Indeksi i fundit është `len(lista) - 1`.

```
my_list = [10, 20, 30, 40]

print(my_list[0], my_list[1], my_list[2], my_list[3])
```

```
10 20 30 40
```

Mund të përdorësh indekset për të aksesuar elementët individualë të një liste. Në shembullin më sipër, `my_list[0]` i referohet elementit të parë (me vlerë `10`), `my_list[1]` i referohet elementit të dytë (me vlerë `20`), etj. Mund të përdorësh indekset në një `while`-cikël për të iteruar mbi elementët e një liste:

```
index = 0
while index < 4:
    print(my_list[index])
    index += 1
```

```
10
20
30
40
```

0	1	2	3
10	20	30	40
-4	-3	-2	-1

Figure 1.5: Indekset pozitive (sipër) dhe negative (poshtë) për `[10, 20, 30, 40]`.

1.5.1 Indekse negative

Mund të përdorësh indekse negative për të numëruar nga fundi i listës: `-1` tregon elementin e fundit, `-2` të parafundit, etj.

```
my_list = [10, 20, 30, 40]
print(my_list[-1], my_list[-2], my_list[-3], my_list[-4])
```

```
40 30 20 10
```

1.5.2 Gabimi `IndexError` kur indeksi është i pavlefshëm

```
my_list = [10, 20, 30, 40]

index = 0
while index < 5: # 5 është jashtë kufijve (indekset vlefshme: 0..3)
    print(my_list[index])
    index += 1
```

```
10
20
30
40
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    print(my_list[index])
IndexError: list index out of range
```

Në iterimin e fundit `index` bëhet `4`, që është indeks i pavlefshëm për këtë listë, ndaj ngrihet `IndexError`. Për të shmangur këtë gabim, sigurohu që indeksi të jetë gjithmonë brenda kufijve të vlefshëm (0 deri në `len(list) - 1` për indekset pozitive, ose `-1` deri në `-len(list)` për indekset negative). `IndexError` është një nga gabimet më të zakonshme në programim, ndaj duhet të jesh i kujdesshëm kur përdor indekse për të aksesuar elementët e një liste.

1.6 Funksioni `len`

`len(seq)` kthen gjatësinë (numrin e elementëve) të një sekuence.

```
my_list = [10, 20, 30, 40]
size = len(my_list) # -> 4
```

`len` përdoret shpesh për të shmangur `IndexError` gjatë iterimit me `while`:

```
my_list = [10, 20, 30, 40]

index = 0
while index < len(my_list):
    print(my_list[index])
    index += 1
```

1.7 Përdorimi i for për të iteruar sipas indeksit

Mund të kombinosh `len` me `range` për të marrë të gjithë indekset e një liste.

```
names = ['Jenny', 'Kelly', 'Chloe', 'Aubrey']

for index in range(len(names)): # -> 0, 1, 2, 3
    print(names[index])
```

```
Jenny
Kelly
Chloe
Aubrey
```

1.8 Listat janë *mutable* (të ndryshueshme)

Listat në Python janë *mutable*, që do të thotë se elementët e tyre mund të ndryshohen. Kjo lejon që një shprehje e formës `list[index]` të shfaqet në anën e majtë të operatorit të caktimit.

```
numbers = [1, 2, 3, 4, 5]
print(numbers)

numbers[0] = 99
print(numbers)
```

```
[1, 2, 3, 4, 5]
[99, 2, 3, 4, 5]
```

Para	1	2	3	4	5
Pas	99	2	3	4	5

Figure 1.6: Ndryshimi i elementit të parë me `numbers[0] = 99`.

Indeks i vlefshëm ose `IndexError`

Gjatë caktimit me indeks duhet të përdorësh një indeks *të vlefshëm* për një element ekzistues, përndryshe ngrihet `IndexError`.

```
numbers = [1, 2, 3, 4, 5] # 5 elementë: indekset 0..4
numbers[5] = 99 # -> IndexError (s'ka element me indeks 5)
```

1.8.1 Mbushja e një liste me indekse

Që të mbushësh një listë me vlera përmes indeksimit, së pari krijoje listën (p.sh. me operatorin e përsëritjes), pastaj kaloi elementët me `for`:

```
numbers = [0] * 5

# Mbush të gjithë elementët me 99
for index in range(len(numbers)):
    numbers[index] = 99

print(numbers) # -> [99, 99, 99, 99, 99]
```

Programi 1 (`sales_list.py`)

```
NUM_DAYS = 5

def main():
    # Krijo një listë për shitjet e çdo dite.
    sales = [0] * NUM_DAYS

    print('Enter the sales for each day.')

    # Merr shitjet për çdo ditë dhe ruaji në listë.
    for index in range(len(sales)):
        sales[index] = float(input(f'Day #{index + 1}: '))

    # Shfaq vlerat e futura.
    print('Here are the values you entered:')
    for value in sales:
        print(value)

if __name__ == '__main__':
    main()
```

```
Enter the sales for each day.
Day #1: 1000
Day #2: 2000
Day #3: 3000
Day #4: 4000
Day #5: 5000
Here are the values you entered:
```

```
1000.0
2000.0
3000.0
4000.0
5000.0
```

1.9 Bashkimi (konkatenimi) i listave

Të konkatenosh do të thotë të bashkosh dy gjëra. Operatorët `+` dhe `+=` bashkojnë lista.

1.9.1 Me + (krijon listë të re)

Me pozicionin e operatorit + midis dy listave, krijohet një listë e re që përmban të gjithë elementët e listës së parë të ndjekur nga të gjithë elementët e listës së dytë. Listat origjinale mbeten të pandryshuara.

```
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
list3 = list1 + list2

print(list1) # [1, 2, 3, 4]
print(list2) # [5, 6, 7, 8]
print(list3) # [1, 2, 3, 4, 5, 6, 7, 8]
```

```
girl_names = ['Joanne', 'Karen', 'Lori']
boy_names = ['Chris', 'Jerry', 'Will']
all_names = girl_names + boy_names
print(all_names)
# ['Joanne', 'Karen', 'Lori', 'Chris', 'Jerry', 'Will']
```

1.9.2 Me += (shton në vend)

Me operatorin +=, elementët e listës së djathtë shtohen në fund të listës së majtë. Kjo *ndryshon* listën e majtë dhe *nuk* krijon një listë të re.

```
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
list1 += list2
print(list1) # [1, 2, 3, 4, 5, 6, 7, 8]
print(list2) # e pandryshuar
```

```
girl_names = ['Joanne', 'Karen', 'Lori']
girl_names += ['Jenny', 'Kelly']
print(girl_names)
# ['Joanne', 'Karen', 'Lori', 'Jenny', 'Kelly']
```

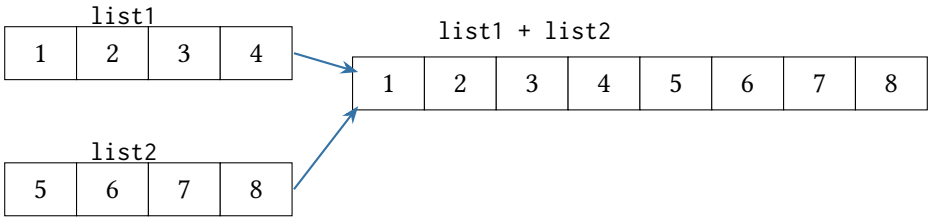

Figure 1.7: Konkatenimi me +: krijon një listë të re me elementët e `list1` të ndjekur nga `list2`.

Kujdes: vetëm listë me listë

Mund të konkatenosh *vetëm* lista me lista. Përpjekja për të konkatenur një listë me një objekt që *nuk* është listë do të ngrejë një përjashtim.

1.10 Prerja e listave (List Slicing)

Përkufizim

Slicing është një shprehje që zgjedh një interval elementësh nga një sekuencë (p.sh. një listë).

Në Python mund të zgjedhësh nënbashkësi (*slices*) nga një listë me formatin e përgjithshëm:

```
list_name[start : end]
```

Ku `start` është indeksi i elementit të parë në slice dhe `end` është indeksi që shënon *fundin* e intervalit (jo i përfshirë). Shprehja kthen një *listë të re* që përmban një kopje të elementëve nga `start` deri *para* `end`.

```
days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday']
mid_days = days[2:5]
print(mid_days)
```

```
['Tuesday', 'Wednesday', 'Thursday']
```

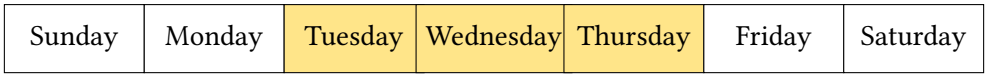

Figure 1.8: Prerja `days[2:5]` merr elementët me indekse 2, 3 dhe 4.

1.10.1 Shembuj të shkurtër me numra

Kur përdorim numra, shprehja `list[start:end]` kthen një listë të re me elementët nga indeksi `start` deri *para* indeksit `end`.

```python
numbers = [1, 2, 3, 4, 5]
print(numbers[1:3]) # -> [2, 3]
```

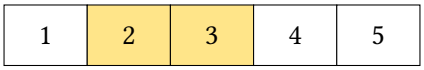

Figure 1.9: Prerja `numbers[1:3]` merr elementët me indekse 1 dhe 2, pra `[2, 3]`.

1.10.2 Lënia bosh e `start` ose `end`

Nëse lë `start` bosh, merret default `0`:

```python
numbers = [1, 2, 3, 4, 5]
print(numbers[:3]) # -> [1, 2, 3]
```

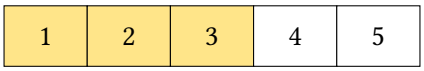

Figure 1.10: Prerja `numbers[:3]` merr elementët me indekse 0, 1 dhe 2, pra `[1, 2, 3]`.

Nëse lë `end` bosh, përdoret gjatësia e listës:

```python
numbers = [1, 2, 3, 4, 5]
print(numbers[2:]) # -> [3, 4, 5]
```

Figure 1.11: Prerja `numbers[2:]` merr elementët nga indeksi 2 deri në fund, pra `[3, 4, 5]`.

Nëse i lë *të dyja* bosh, merr një kopje të të gjithë listës:

```python
numbers = [1, 2, 3, 4, 5]
print(numbers[:]) # -> [1, 2, 3, 4, 5]
```

Figure 1.12: Prerja `numbers[:]` merr të gjithë elementët e listës, pra `[1, 2, 3, 4, 5]`.

1.10.3 Hapi i prerjes (opsional)

Slicing mund të ketë edhe *hapin* (`step`) për të anashkaluar elementë. `step` do të jetë numri i elementëve që do të anashkalohen midis atyre që përfshihen në slice.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(numbers[1:8:2]) # -> [2, 4, 6, 8]
```

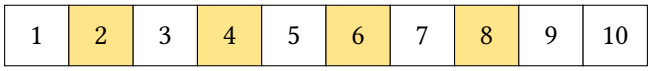

Figure 1.13: Prerja `numbers[1:8:2]` merr çdo të dytin element në intervalin.

1.10.4 Indekse negative në slicing

Indekset negative referojnë pozicione relativisht ndaj fundit të listës:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(numbers[-5:]) # -> [6, 7, 8, 9, 10]
```

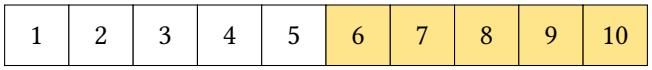

Figure 1.14: Prerja `numbers[-5:]` merr pesë elementët e fundit të listës: `[6, 7, 8, 9, 10]`.

Sjellja kur indekset dalin jashtë kufijve

- Nëse `end` është përtej fundit të listës, Python përdor gjatësinë e listës.
- Nëse `start` është para fillimit të listës, Python përdor `0`.
- Nëse `start > end`, shprehja e slicing kthen një listë bosh `[]`.

1.11 Kërkimi i elementëve në lista me operatorin `in`

Përkufizim

Me operatorin `in` mund të kontrollosh nëse një element ndodhet brenda një liste.

Forma e përgjithshme:

```
item in list # -> True ose False
```

Nëse `item` gjendet në `list`, shprehja kthen `True`, përndryshe `False`.

Programi 2 (`in_list.py`)

```
def main():
    prod_nums = ['V475', 'F987', 'Q143', 'R688']

```

```
    search = input('Enter a product number: ')

    # Kontrollo nëse numri i produktit gjendet në listë.
    if search in prod_nums:
        print(f'{search} was found in the list.')
    else:
        print(f'{search} was not found in the list.')

if __name__ == '__main__':
    main()
```

```
Enter a product number: Q143
Q143 was found in the list.
```

```
Enter a product number: B000
B000 was not found in the list.
```

Ekziston edhe forma e kundërt `not in`, e cila kthen `True` kur elementi *nuk* gjendet në listë:

```
if search not in prod_nums:
    print(f'{search} was not found in the list.')
else:
    print(f'{search} was found in the list.')
```

1.12 Kopjimi i listave — pesë mënyra

1.12.1 Pse `list1 = list2` nuk është kopjim?

Në Python, shprehja `list1 = list2` **nuk** krijon një kopje të re të listës. Kjo thjesht bën që të dyja variablat `list1` dhe `list2` të referojnë të *njëjtën* listë në memorje. Çdo ndryshim i bërë përmes njërës variabël do të reflektohet edhe te tjetra.

```
list2 = [1, 2, 3]
list1 = list2
list1[0] = 99
print(list1)
print(list2)
```

```
[99, 2, 3]
[99, 2, 3]
```

Siç shihet, ndryshimi i `list1` ndryshon edhe `list2`, sepse të dyja referojnë të njëjtin objekt. Për të krijuar një *kopje të pavarur*, përdor një nga mënyrat e mëposhtme: `copy()`, `list()`, `[:]`, etj.

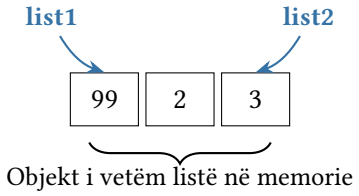

Figure 1.15: Të dyja `list1` dhe `list2` referojnë të njëjtën listë në memorie.

1.12.2 Mënyra 1 — `list.copy()`

```
list1 = [1, 2, 3]
list3 = list1.copy()
print("Lista 3:", list3)
```

- `list3 = list1.copy()` krijon një listë të re me të njëjtat elemente si `list1`.
- **Tip**: është mënyra më e drejtpërdrejtë për të kopjuar një listë ekzistuese.

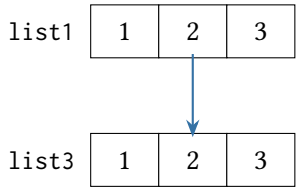

Figure 1.16: Kopjimi i listës me `copy()`: `list3` është kopje e pavarur e `list1`.

1.12.3 Mënyra 2 — `list(iterable)`

```
list1 = [1, 2, 3]
list4 = list(list1)
print("Lista 4:", list4)
```

- `list4 = list(list1)` konverton çdo *iterable* në listë; këtu prodhon një kopje të `list1`.
- **Përfitim**: funksionon edhe kur burimi nuk është listë (p.sh. `range`, `tuple`, etj.).

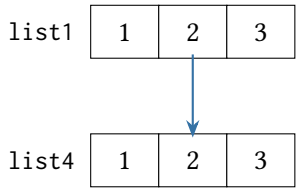

Figure 1.17: Kopjimi me `list()`: `list4` është kopje e re e `list1`.

1.12.4 Mënyra 3 — `[:]` (slicing i plotë)

```
list1 = [1, 2, 3]
list5 = list1[:]
print("Lista 5:", list5)
```

- `list5 = list1[:]` kthen një kopje të të gjithë elementëve nga fillimi në fund.
- **Shkurt**: sintaksë e shkurtër, shumë e përdorur.

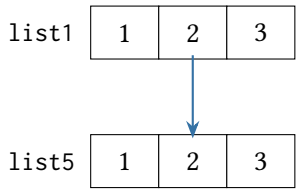

Figure 1.18: Kopjimi me slicing: `list5 = list1[:]` prodhon një listë të re.

1.12.5 Mënyra 4 — `for` + `append`

```
list1 = [1, 2, 3]
list6 = []
for i in list1:
        list6.append(i)
print("Lista 6:", list6)
```

- Ndërtim "me dorë": krijon një listë bosh dhe shton elementët një e nga një.
- **Fleksibel**: mund të aplikosh transformime gjatë kopjimit (p.sh. `append(i * 2)`).

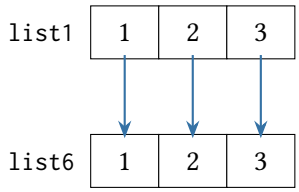

Figure 1.19: Kopjimi me `for` dhe `append`: elementët shtohen një nga një në listën e re.

1.12.6 Mënyra 5 — `[] + list`

```
list1 = [1, 2, 3]
list7 = [] + list1
print("Lista 7:", list7)
```

- `list7 = [] + list1` përfiton nga konkatenimi: një listë bosh + `list1` = kopje e `list1`.
- **Efekt**: prodhon listë të re, pa prekur `list1`.

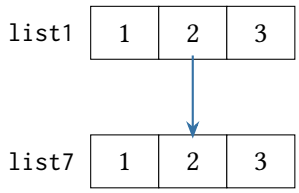

Figure 1.20: Kopjimi me konkatenim: `list7 = [] + list1` prodhon një listë të re.

E rëndësishme

Të gjitha mënyrat e mësipërme prodhojnë **kopje të cekëta** (*shallow copies*). Kjo do të thotë se nëse elementët janë objekte të përbëra (p.sh. nën-lista), referencat e tyre ricopjohen, por objektet e brendshme nuk dubliohen.

1.13 Përpunimi i listave

Deri tani kemi parë shumë teknika për të punuar me lista. Këtu përmbledhim disa modele të zakonshme për *përpunimin* e të dhënave brenda listave: llogaritja e shumës, mesatarja, kalimi i listave si argumente dhe kthimi i listave nga funksionet.

1.13.1 Shuma e vlerave në një listë

Përkufizim

Akumulatori është një variabël që "mbledh" gradualisht rezultatet gjatë iterimit (p.sh. shuma).

Programi 3 (`total_list.py`)

```python
def main():
    numbers = [2, 4, 6, 8, 10]

    # Variabël akumuluese.
    total = 0

    # Mblidh vlerat e listës.
    for value in numbers:
        total += value

    print(f'The total of the elements is {total}.')

if __name__ == '__main__':
    main()
```

```
The total of the elements is 30.
```

Shkurtesë e dobishme

Në vend të ciklit mund të përdorësh `sum(numbers)` për të marrë shumën. Kompleksiteti kohor mbetet `O(n)`.

1.13.2 Mesatarja e vlerave në një listë

Mesatarja arrihet duke pjestuar shumën me numrin e elementëve.

Programi 4 (average_list.py)

```
def main():
    scores = [2.5, 7.3, 6.5, 4.0, 5.2]

    total = 0.0
    for value in scores:
        total += value

    average = total / len(scores)
    print(f'The average of the elements is {average}.')

if __name__ == '__main__':
    main()
```

Rast i veçantë: listë bosh

Nëse lista është bosh, `len(lista) == 0` dhe pjestimi shkakton `ZeroDivisionError`. Zgjidhje e thjeshtë:

```
average = (sum(scores) / len(scores)) if scores else 0.0
```

Për të dhëna *float* mund të përdorësh edhe `statistics.fmean(scores)` nga moduli `statistics`.

1.13.3 Kalimi i një liste si argument funksioni

Funksionet bëjnë programin më të kuptueshëm dhe të mirëmbajtshëm. Mund të kalosh një listë si argument te një funksion që kryen një operacion (p.sh. totalin).

Programi 5 (total_function.py)

```
def main():
    numbers = [2, 4, 6, 8, 10]
    print(f'The total is {get_total(numbers)}.')

# get_total pranon një listë dhe kthen shumën e elementëve.
def get_total(value_list):
    total = 0
    for num in value_list:
        total += num
    return total

if __name__ == '__main__':
    main()
```

E rëndësishme: *mutability*

Kur kalon një listë te një funksion, kalon *referencën*. Nëse funksioni e ndryshon listën (p.sh. `value_list.append(99)`), ndryshimi reflektohet edhe jashtë funksionit. Nëse s'dëshiron këtë sjellje, puno me një *kopje* (p.sh. `value_list.copy()`).

1.13.4 Kthimi i një liste nga një funksion

Një funksion mund të krijojë dhe të kthejë një listë, për ta përdorur më pas në pjesë të tjera të programit.

Programi 6 (`return_list.py`)

```
def main():
    # Merr një listë me vlera të futura nga përdoruesi.
    numbers = get_values()

    print('The numbers in the list are:')
    print(numbers)

# get_values lexon disa numra dhe i kthen si listë.
def get_values():
    values = []
    again = 'y'

    while again.lower() == 'y':
        num = int(input('Enter a number: '))
        values.append(num)

        print('Do you want to add another number?')
        again = input('y = yes, anything else = no: ')
        print()

    return values

if __name__ == '__main__':
    main()
```

```
Enter a number: 1
Do you want to add another number?
y = yes, anything else = no: y
Enter a number: 2
Do you want to add another number?
y = yes, anything else = no: y
Enter a number: 3
Do you want to add another number?
y = yes, anything else = no: n

The numbers in the list are:
[1, 2, 3]
```

1.14 Përzgjedhja e rastësishme e elementëve të një liste

Moduli `random` ofron mënyra të thjeshta për të zgjedhur elementë rastësisht nga një listë.

1.14.1 `random.choice` — një element i vetëm

Metoda `random.choice(list)` zgjedh një element rastësisht nga `list`.

```
import random

names = ['Jenny', 'Kelly', 'Chloe', 'Aubrey']
winner = random.choice(names)
print(winner)
```

```
Kelly
```

1.14.2 `random.choices` — disa elementë (me përsëritje të mundshme)

Metoda `random.choices(list, k=n)` zgjedh `n` elementë rastësisht nga `list`, duke lejuar dublikatat.

```
import random

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
selected = random.choices(numbers, k=3)
print(selected)
```

```
[8, 7, 7]
```

Pesha opsionale

`random.choices` pranon edhe `weights=` ose `cum_weights=` për të favorizuar disa elementë.

1.14.3 `random.sample` — elementë unikë (pa përsëritje)

```
import random

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
selected = random.sample(numbers, k=3)
print(selected) # p.sh. [4, 10, 2]
```

1.15 Punë me lista dhe skedarë

Shpesh duhet të ruash përmbajtjen e një liste në skedar, apo të lexosh nga skedari në listë.

Kur përdorim lista me skedarë

Shpesh, të dhënat që ruhen në listë duhen ruajtur në disk për përdorim të mëvonshëm, ose duhen lexuar nga një skedar për t'u përpunuar në program. Kjo është veçanërisht e dobishme për ruajtjen e të dhënave të përdoruesit, konfigurimeve, rezultateve të llogaritjeve, etj.

1.15.1 Shkrimi i një liste *string*-ësh: `writelines`

Përkufizim

`writelines(lista)` është një metodë e objektit skedar që shkruan të gjithë elementët e listës në skedar, njëri pas tjetrit, **pa** shtuar automatikisht ndarës rreshti (`\n`).

Programi 7 (`writelines.py`)

```
def main():
    cities = ['New York', 'Boston', 'Atlanta', 'Dallas']
    with open('cities.txt', 'w', encoding='utf-8') as outfile:
        outfile.writelines(cities) # -> të gjitha në një rresht!

if __name__ == '__main__':
    main()
```

Përmbajtja e skedarit `cities.txt` pas ekzekutimit:

```
New YorkBostonAtlantaDallas
```

Pse? Sepse `writelines` shkruan *ashtu siç është* në listë, pa shtuar asgjë midis elementëve.

Kujdes

`writelines` **nuk** shton ' `\n` ' vetë, prandaj elementët do të bashkohen në një rresht.

1.15.2 Shkrimi i një elementi në çdo rresht

Përkufizim

Për të shkruar çdo element të listës në një rresht të veçantë në skedar, duhet të shtosh manualisht karakterin e ri-rreshtit (`\n`) pas çdo elementi.

Programi 8 (`write_list.py`)

```
def main():
    cities = ['New York', 'Boston', 'Atlanta', 'Dallas']
    with open('cities.txt', 'w', encoding='utf-8') as outfile:
        for item in cities:
            outfile.write(item + '\n')
```

```python

if __name__ == '__main__':
    main()
```

Përmbajtja e skedarit `cities.txt` pas ekzekutimit:

```
New York
Boston
Atlanta
Dallas
```

Nëse përdor `print(item, file=outfile)`, `print` shton automatikisht ' `\n` ' në fund të çdo thirrjeje.

Programi 9 (`write_list_print.py`)

```python
def main():
    cities = ['New York', 'Boston', 'Atlanta', 'Dallas']
    with open('cities.txt', 'w', encoding='utf-8') as outfile:
        for item in cities:
            print(item, file=outfile)

if __name__ == '__main__':
    main()
```

Pse përdorim `print` me `file=outfile`?

Funksioni `print` është më i thjeshtë për të shtuar automatikisht ndarësin e rreshtit dhe është më i sigurt për të shmangur gabime të zakonshme.

Çfarëdo mënyre të përdorësh, rezultati në skedar do të jetë i njëjtë. Përdorimi i `print` është më i thjeshtë dhe më pak i prirur për gabime.

1.15.3 Leximi në listë me `readlines` dhe `rstrip`

Përkufizim

`readlines()` lexon të gjithë rreshtat nga një skedar tekst dhe i kthen si listë *string*-ësh, ku secili element përmban një rresht (përfshirë karakterin ' `\n` ' në fund).

Programi 10 (`read_list.py`)

```python
def main():
    with open('cities.txt', 'r', encoding='utf-8') as infile:
        cities = infile.readlines()

    # Hiq '\n' nga çdo element
    for i in range(len(cities)):
        cities[i] = cities[i].rstrip('\n')
```

```python

    print(cities)

if __name__ == '__main__':
    main()
```

Pse përdorim `rstrip`?

Kur lexon rreshta nga skedari, secili element i listës përfundon me ' `\n` '. `rstrip('\n')` heq këtë karakter që të marrësh vetëm tekstin e pastër.

1.15.4 Shkrimi i numrave në skedar

Përkufizim

Kur shkruan numra në skedar tekst, duhet t'i konvertosh në *string* (me `str()` ose `f-string`) dhe të shtosh ' `\n` ' për çdo rresht.

Programi 11 (`write_number_list.py`)

```python
def main():
    numbers = [1, 2, 3, 4, 5, 6, 7]
    with open('numberlist.txt', 'w', encoding='utf-8') as outfile:
        for item in numbers:
            outfile.write(f'{item}\n') # ose str(item) + '\n'

if __name__ == '__main__':
    main()
```

1.15.5 Leximi i numrave dhe konvertimi në `int`

Përkufizim

Kur lexon numra nga skedari, ata vijnë si *string*-je. Duhet t'i konvertosh në `int` ose `float` për t'i përdorur si numra në program.

Programi 12 (`read_number_list.py`)

```python
def main():
    with open('numberlist.txt', 'r', encoding='utf-8') as infile:
        numbers = infile.readlines()

    for i in range(len(numbers)):
        numbers[i] = int(numbers[i]) # heq '\n' automatikisht gjatë int()

    print(numbers)

if __name__ == '__main__':
    main()
```

Përmbledhje

- Për të ruajtur një listë në skedar, përdor `write`, `writelines`, ose `print(..., file=...)`.
- Për të lexuar një listë nga skedari, përdor `readlines()` dhe pastro elementët me `rstrip()` ose konvertoji në tipin e duhur.
- Kujdes me ndarësit e rreshtave dhe konvertimin e tipeve!

1.16 List Comprehensions (Përftimi i listave)

Përkufizim

List comprehension është një shprehje e shkurtër që krijon një listë të re duke *iteruar* mbi elementët e një liste ekzistuese dhe duke prodhuar një rezultat për çdo element.

1.16.1 Pse t'i përdorim?

Shpesh kemi nevojë të lexojmë një listë hyrëse dhe prej saj të prodhojmë një listë dalëse. Në vend të një `for`-cikli me `append`, përdorim një shprehje kompakte.

```
[result_expression iteration_expression]
```

Ku `result_expression` është vlera që futet në listën e re për çdo iterim dhe `iteration_expression` ka formën `for target in iterable`.

Shembull — Kopjimi i një liste

Me `for` + `append`:

```
list1 = [1, 2, 3, 4]
list2 = []
for item in list1:
    list2.append(item)
```

E njëjta gjë me list comprehension:

```
list1 = [1, 2, 3, 4]
list2 = [item for item in list1]
```

1.16.2 Shembuj transformimi

Shembull — Katrorët e elementëve

Me `for`:

```
list1 = [1, 2, 3, 4]
list2 = []
for item in list1:
    list2.append(item**2)
```

Me list comprehension:

```
list1 = [1, 2, 3, 4]
list2 = [item**2 for item in list1]
print(list2)
```

```
[1, 4, 9, 16]
```

Shembull — Gjatësitë e stringjeve

Me `for`:

```
str_list = ['Winken', 'Blinken', 'Nod']
len_list = []
for s in str_list:
    len_list.append(len(s))
```

Me list comprehension:

```
str_list = ['Winken', 'Blinken', 'Nod']
len_list = [len(s) for s in str_list]
print(len_list)
```

```
[6, 7, 3]
```

1.16.3 Filtrim me klauzolën `if`

Mund të shtosh një `if`-klauzolë në fund për të *filtruar* elementët hyrës.

Përkufizim — Formati me filtër

```
[result_expression iteration_expression if_clause]
```

Shembull — Zgjedh numrat < 10

Me for:

```
list1 = [1, 12, 2, 20, 3, 15, 4]
list2 = []
for n in list1:
    if n < 10:
        list2.append(n)
```

Me list comprehension:

```
list1 = [1, 12, 2, 20, 3, 15, 4]
list2 = [item for item in list1 if item < 10]
print(list2)
```

```
[1, 2, 3, 4]
```

Shembull — Filtro sipas gjatësisë së emrit

```
last_names = ['Jackson', 'Smith', 'Hildebrandt', 'Jones']
short_names = [name for name in last_names if len(name) < 6]
print(short_names)
```

```
['Smith', 'Jones']
```

Këshilla praktike

- Prefero list comprehensions kur transformimi/filtrimi është *i thjeshtë dhe i lexueshëm.* Për logjikë komplekse, një `for`-cikël i zakonshëm është më i qartë.
- Kompleksiteti kohor është `O(n)` si te cikli; shpesh janë më të shpejta sepse implementohen më afër C-së.
- Mund të përdorësh edhe operatorin `if ... else` *brenda* `result_expression` (p.sh. `[x**2 if x>0 else 0 for x in nums]`).

1.17 Metodat e listave dhe funksione të dobishme

Përkufizim

Listat ofrojnë metoda për të *shtuar*, *hequr*, *renditur*, *kthyer mbrapsht* elementët, etj. Python ka edhe funksione të brendshme si `min` dhe `max` që punojnë mbi sekuenca.

Disa metoda të zakonshme

`append(item)`	Shton `item` në fund të listës.
`index(item)`	Kthen indeksin e paraqitjes së parë të `item` (hedh `ValueError` nëse s'gjendet).
`insert(index, item)`	Fut `item` në pozicionin `index`, duke zhvendosur të tjerët djathtas.
`sort()`	Rendit elementët në rritje (ascending).
`remove(item)`	Heq paraqitjen e parë të `item` (hedh `ValueError` nëse s'gjendet).
`reverse()`	Përmbys rendin e elementëve në vend.

1.17.1 `append` — shto në fund

append është metoda më e përdorur për të shtuar një element të ri në fund të një liste ekzistuese. Kjo metodë ndryshon listën në vend (*in place*) dhe nuk kthen vlerë. Sintaksa është `lista.append(vlera)`. Pas thirrjes, elementi i ri vendoset në pozicionin e fundit të listës.

```
nums = [1, 2, 3]
nums.append(4)
print(nums) # [1, 2, 3, 4]
```

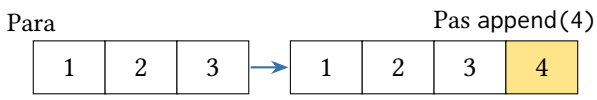

Figure 1.21: Shtimi i një elementi në fund me append.

1.17.2 `index` — gjej pozicionin e një elementi

Metoda **index** kërkon për një vlerë të caktuar në listë dhe kthen indeksin e paraqitjes së parë të saj. Nëse elementi nuk gjendet, ngrihet përjashtimi `ValueError`. Kjo është e dobishme për të gjetur pozicionin e një elementi për ta ndryshuar ose për ta përdorur më tej.

```
food = ['Pizza', 'Burgers', 'Chips']
pos = food.index('Burgers') # -> 1
food[pos] = 'Pickles'
print(food) # ['Pizza', 'Pickles', 'Chips']
```

Gabimi `ValueError`

`list.index(x)` hedh `ValueError` nëse `x` nuk gjendet në listë. Përdor `if x in lista: ...` përpara, ose `try/except`.

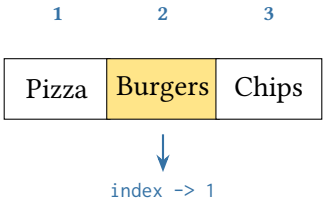

Figure 1.22: `index('Burgers')` kthen 1.

1.17.3 `insert` — fut në pozicion dhe zhvendos djathtas

Metoda **insert** lejon të shtosh një element në një pozicion të caktuar të listës. Elementët që ndodhen në atë pozicion dhe më tej zhvendosen një pozicion djathtas. Sintaksa është `lista.insert(indeks, vlera)`.

```
names = ['James', 'Kathryn', 'Bill']
names.insert(0, 'Joe')
print(names) # ['Joe', 'James', 'Kathryn', 'Bill']
```

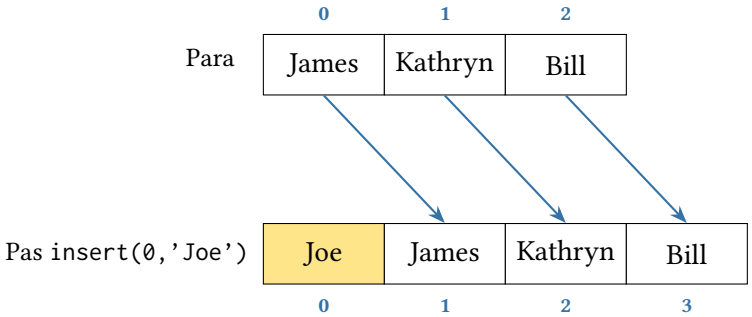

Figure 1.23: Futja në indeksin 0 zhvendos elementët ekzistues një pozicion djathtas.

Indekse jashtë kufijve

`insert` nuk hedh përjashtim për indekse "të këqija": indeksi >= gjatësisë fut në fund; një indeks negativ shumë i vogël fut në fillim.

1.17.4 `sort` — rendit në rritje

Metoda **sort** rendit elementët e listës në mënyrë rritëse (nga më i vogli te më i madhi). Kjo metodë ndryshon listën ekzistuese dhe nuk kthen vlerë. Për të marrë një listë të re të renditur, përdor funksionin `sorted(lista)`.

```
a = [9, 1, 0, 2, 8, 6, 7, 4, 5, 3]
print('Orig:', a)
a.sort()
print('Sort:', a)
```

Para

9	1	0	2	8	6	7	4	5	3
0	1	2	3	4	5	6	7	8	9

Pas `sort()`

Figure 1.24: Renditja nga më i vogli te më i madhi.

Renditje *in place*

`list.sort()` ndryshon listën në vend. Për të marrë një kopje të renditur, përdor `sorted(lista)` që kthen listë të re.

1.17.5 `remove` — hiq paraqitjen e parë

Metoda **remove** heq nga lista paraqitjen e parë të një elementi të caktuar. Nëse elementi nuk gjendet, ngrihet përjashtimi `ValueError`. Elementët që ndodhen pas tij zhvendosen një pozicion majtas.

```
food = ['Pizza', 'Burgers', 'Chips', 'Burgers']
food.remove('Burgers')
print(food) # ['Pizza', 'Chips', 'Burgers']
```

Nëse s'gjendet

`remove(x)` hedh `ValueError` nëse `x` nuk është në listë.

Para

Pizza	Burgers	Chips	Burgers

Pas `remove('Burgers')`

Pizza	Chips	Burgers

Figure 1.25: Hiqet vetëm paraqitja e parë; elementët pas saj zhvendosen majtas.

1.17.6 `reverse` — kthe rendin mbrapsht

Metoda **reverse** përmbys rendin e elementëve të listës ekzistuese. Kjo është e dobishme kur dëshiron të lexosh ose të përpunosh listën nga fundi në fillim.

```
a = [1, 2, 3, 4, 5]
a.reverse()
print(a) # [5, 4, 3, 2, 1]
```

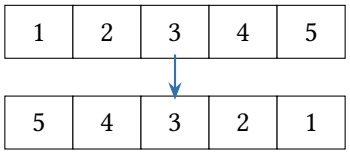

Figure 1.26: `reverse()` përmbys rendin *in place*.

1.17.7 `del` — fshi sipas indeksit

Fjala kyçe **del** përdoret për të fshirë një element nga lista sipas indeksit të tij. Pas fshirjes, elementët që ndodhen pas tij zhvendosen një pozicion majtas.

```
a = [1, 2, 3, 4, 5]
del a[2]
print(a) # [1, 2, 4, 5]
```

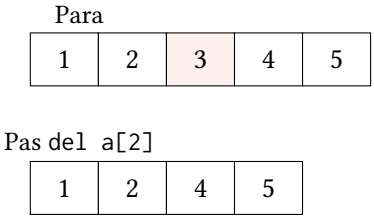

Figure 1.27: Fshirja me `del` sipas indeksit; elementët pas tij zhvendosen majtas.

1.17.8 `min` dhe `max` — vlerat skajore

Funksionet **min** dhe **max** kthejnë përkatësisht vlerën më të vogël dhe më të madhe nga elementët e listës. Këto funksione janë të dobishme për të gjetur kufijtë e të dhënave në një listë numerike ose të renditshme.

```
vals = [5, 4, 3, 2, 50, 40, 30]
print(min(vals)) # 2
print(max(vals)) # 50
```

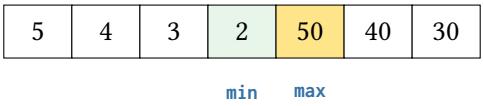

Figure 1.28: `min` dhe `max` gjejnë vlerat skajore në listë.

1.18 Listat dy-dimensionale (Two-Dimensional Lists)

Përkufizim

Listë dy-dimensionale quajmë një listë ku *çdo element është vetë listë.* Mendohet si një tabelë me *rreshta* dhe *kollona.*

Një listë dy-dimensionale është një strukturë të dhënash që mund të përfytyrohet si një tabelë ose matricë, ku të dhënat janë të organizuara në rreshta dhe kolona. Në Python, kjo realizohet duke pasur një listë, ku çdo element i saj është vetë një listë tjetër. Kjo lejon ruajtjen dhe përpunimin e të dhënave në formë tabelare, si p.sh. nota të nxënësve, të dhëna të një loje, ose vlera të një matrice matematikore.

1.18.1 Shembull i shpejtë: listë me emra (3×2)

```
students = [['Joe', 'Kim'],
            ['Sam', 'Sue'],
            ['Kelly', 'Chris']]

print(students)
print(students[0])
print(students[1])
print(students[2])
```

```
[['Joe','Kim'], ['Sam','Sue'], ['Kelly','Chris']]
['Joe', 'Kim']
['Sam', 'Sue']
['Kelly', 'Chris']
```

	Kolona 0	Kolona 1
Rreshti 0	Joe	Kim
Rreshti 1	Sam	Sue
Rreshti 2	Kelly	Chris

Figure 1.29: Lista `students` si tabelë 3×2 (rreshta & kolona të numëruara nga 0).

1.18.2 Indeksimi me dy nënindekse

Elementet adresohen me `lista[rreshti][kolona]`. Për të aksesuar një element të caktuar në një listë dy-dimensionale, përdor dy indekse: i pari tregon rreshtin, i dyti kolonën. Për shembull, `lista[1][2]` i referohet elementit në rreshtin 1 dhe kolonën 2. Kjo është e ngjashme me mënyrën si adresohen qelizat në një tabelë ose matricë.

```
scores = [[0, 0, 0],
          [0, 0, 0],
          [0, 0, 0]]

# Elementët e rreshtit 0:
```

```
scores[0][0]; scores[0][1]; scores[0][2]

# Elementët e rreshtit 1:
scores[1][0]; scores[1][1]; scores[1][2]

# Elementët e rreshtit 2:
scores[2][0]; scores[2][1]; scores[2][2]
```

	Kolona 0	Kolona 1	Kolona 2
Rreshti 0	scores[0][0]	scores[0][1]	scores[0][2]
Rreshti 1	scores[1][0]	scores[1][1]	scores[1][2]
Rreshti 2	scores[2][0]	scores[2][1]	scores[2][2]

Figure 1.30: Nënindekset për çdo element të `scores` (3×3).

1.18.3 Iterimi: dy `for`-cikla të folezuar (row-major)

Për të përpunuar të gjithë elementët e një liste dy-dimensionale, përdoren zakonisht dy cikle `for` të folezuar: cikli i jashtëm kalon nëpër rreshta, ndërsa cikli i brendshëm kalon nëpër kolonat e çdo rreshti. Ky model quhet *row-major order* dhe është mënyra më e zakonshme për të lexuar ose ndryshuar të dhënat në një tabelë.

Programi 13 (`two_dimensional_list.py`)

```python
def main():
    # Krijo një listë 3x3.
    values = [[1, 2, 3],
              [10, 20, 30],
              [100, 200, 300]]

    # Shfaq elementët (rresht pas rreshti).
    for row in values:
        for element in row:
            print(element)

if __name__ == '__main__':
    main()
```

	Kolona 0	Kolona 1	Kolona 2
Rreshti 0	1	2	3
Rreshti 1	10	20	30
Rreshti 2	100	200	300

Figure 1.31: Elementët e `values` (3×3) me nënindekset përkatëse.

1.18.4 Caktimi i vlerave me indekse

Për të ndryshuar vlerat në një listë dy-dimensionale, përdor dy cikle `for` për të kaluar nëpër të gjithë rreshtat dhe kolonat, dhe cakto vlerën e re me `lista[rreshti][kolona] = vlerë`. Kjo është mënyra standarde për të mbushur ose modifikuar një tabelë me të dhëna.

Programi 14 (`random_numbers.py`)

```
# Cakton numra të rastësishëm në një listë 2D.
import random

ROWS = 3
COLS = 4

def main():
    values = [[0 for _ in range(COLS)] for _ in range(ROWS)]

    # Mbush me numra të rastësishëm.
    for r in range(ROWS):
        for c in range(COLS):
            values[r][c] = random.randint(1, 100)

    print(values)

if __name__ == '__main__':
    main()
```

Kujdes me krijimin e listës 2D

`values = [[0]*COLS]*ROWS` krijon *të njëjtin* rresht të përsëritur (referenca të përbashkëta). Përdor gjithmonë list comprehension me dy nivele si më sipër që çdo rresht të jetë i pavarur.

Disa operacione të zakonshme me lista dy-dimensionale përfshijnë: llogaritjen e shumës së të gjithë elementëve, marrjen e një rreshti të caktuar si listë të veçantë, ose marrjen e një kolone të caktuar. Këto modele janë të dobishme për përpunim të dhënash tabelare.

```
total = sum(values[r][c] for r in range(ROWS) for c in range(COLS))

# Merr rreshtin r (si listë)
row_r = values[r]

# Merr kolonën c (si listë)
col_c = [values[r][c] for r in range(ROWS)]
```

1.19 Tuplet

Përkufizim

Tuple është një *sekuencë* shumë e ngjashme me listën, por **immutable** (e pandryshueshme): pasi krijohet, përmbajtja *nuk* mund të ndryshohet.

Një tuple është një koleksion i renditur elementësh, të cilët mund të jenë të tipeve të ndryshme (numra, stringje, etj.), por ndryshe nga lista, nuk mund të ndryshohet pas krijimit. Kjo do të thotë se nuk mund të shtosh, heqësh ose ndryshosh elementë të tuplet. Tuplet janë të dobishme kur dëshiron të ruash të dhëna që nuk duhet të ndryshohen gjatë ekzekutimit të programit.

1.19.1 Krijimi dhe shfaqja

Tuplet krijohen me kllapa të rrumbullakëta `( ... )`, por Python i lejon edhe pa kllapa në shumë raste, nëse elementët ndahen me presje.

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple)

another_tuple = 10, 20, 30
print(another_tuple)
```

```
(1, 2, 3, 4, 5)
(10, 20, 30)
```

1.19.2 Iterimi me `for`

Ashtu si listat, tuplet mund të iterohen direkt.

```
names = ('Holly', 'Warren', 'Ashley')
for n in names:
    print(n)
```

```
Holly
Warren
Ashley
```

1.19.3 Indeksimi dhe `range(len(...))`

Tuplet mbështesin indeksimin (nga `0` si zakonisht).

```
names = ('Holly', 'Warren', 'Ashley')
for i in range(len(names)):
    print(names[i])
```

```
Holly
Warren
Ashley
```

1.19.4 Çfarë mbështesin tuplet

Tuplet mbështesin pothuajse të gjitha operacionet e listave që *nuk* ndryshojnë përmbajtjen: mund të lexosh elementë me indeks, të marrësh një pjesë të tuplet me slicing, të përdorësh operatorët `in` / `not in`, të bashkosh tuplet me `+` ose të përsëritësh me `*`. Gjithashtu, mund të përdorësh funksionet `len`, `min`, `max`, dhe metodën `index` për të gjetur pozicionin e një vlere.

1.19.5 Çfarë *nuk* mbështesin tuplet

Ndryshe nga listat, tuplet nuk kanë metoda që ndryshojnë përmbajtjen, si `append`, `remove`, `insert`, `reverse`, `sort`. Çdo përpjekje për caktim me indeks shkakton `TypeError`.

```
t = (1, 2, 3)
t[0] = 99
```

```
TypeError: 'tuple' object does not support item assignment
```

1.19.6 Tuple me një element

Për të krijuar një tuple me një element të vetëm, duhet të përdorësh një presje pas vlerës, përndryshe Python e konsideron si vlerë të zakonshme, jo tuple.

```
one_el = (1,) # tuple me një element
not_tuple = (1) # thjesht numri 1 (jo tuple)
```

1.19.7 Pse të përdorim tuplet?

Tuplet janë të dobishme në situata kur dëshiron të ruash të dhëna që nuk duhet të ndryshohen, si koordinata (x, y), data, ose rezultate që nuk duhet të modifikohen aksidentalisht. Përveç kësaj, tuplet janë më të shpejta se listat për përpunim, sepse janë struktura më të thjeshta dhe të pandryshueshme. Në shumë raste, Python i përdor tuplet për të kthyer disa vlera nga një funksion, ose për të kaluar të dhëna të pandryshueshme si argument.

1.19.8 Konvertimi mes listave dhe tupleve

Përdor `list()` për të kthyer një tuple në listë, dhe `tuple()` për të kthyer një listë në tuple.

```
number_tuple = (1, 2, 3)
number_list = list(number_tuple)
print(number_list)

str_list = ['one', 'two', 'three']
str_tuple = tuple(str_list)
print(str_tuple)
```

```
[1, 2, 3]
('one', 'two', 'three')
```

1.19.9 Shembuj përdorimi të tupleve

Tuplet përdoren shpesh për të ruajtur të dhëna të lidhura që nuk duhet të ndryshohen, si p.sh. koordinata gjeografike, data, ose rezultate të pandryshueshme nga funksionet. Për shembull:

```
point = (4, 5)
date = (2024, 6, 1)
```

Gjithashtu, funksionet mund të kthejnë disa vlera njëherësh duke përdorur tuple:

```
def min_max(values):
    return (min(values), max(values))

nums = [3, 7, 2, 9]
lo, hi = min_max(nums)
print(lo, hi)
```

```
2 9
```

Në këtë shembull, funksioni `min_max` kthen një tuple me vlerat minimale dhe maksimale nga lista e dhënë. Kjo është një mënyrë e thjeshtë dhe e qartë për të kthyer shumë vlera nga një funksion. Variabli `lo` merr vlerën minimale dhe `hi` vlerën maksimale.

Çfarë mësuam

- **Listat** janë të ndryshueshme; **tuples** jo. Të dyja janë sekuenca me rend.
- Indeksimi fillon nga `0`; indekset `-1,-2,...` kapin elementët nga fundi. `len(seq)` jep numrin e elementëve.
- **Slicing** kthen *kopje* të një intervali; `[:]` kopjon të gjithë listën; `step` lejon anashkalime; indekset jashtë kufijve normalizohen.
- Operatorët: `+` / `+=` për *konkatenim*; `*` për *përsëritje*; `in` / `not in` për kërkim.
- **Kopjimi i listave**: `copy()`, `list(...)`, `[:]`, `for+append`, `[] + list` — të gjitha bëjnë *shallow copy*.
- **Metodat kryesore**: `append`, `insert`, `index`, `sort`, `remove`, `reverse`; fshirja me **del**. `min` / `max` për vlerat skajore.
- **Modelet e përpunimit**: shuma/mesatarja (kujdes listat bosh), kalimi i listave si argument dhe kthimi i tyre nga funksionet.
- **List comprehensions**: sintaksë e shkurtër për transformim dhe filtër; e qartë kur logjika është e thjeshtë.
- **Skedarët**: `writelines` nuk shton `\n`; `write` / `print` për një element në çdo rresht; `readlines` + `rstrip` për pastrim; konvertime të tipeve gjatë leximit.
- **Lista 2D**: adresimi `[r][c]`, iterim me dy `for`-cikla; krijo rreshta të pavarur me `[[0 for _ in range(C)] for _ in range(R)]`.
- Tuplet: sekuenca e pandryshueshme; krijo me `(...)` ose `...,...`; mbështesin indeksim, slicing, `in` / `not in`, `+`, `*`, `len`, `min`, `max`, `index`.